

Semana 4

Treinar o Transformer

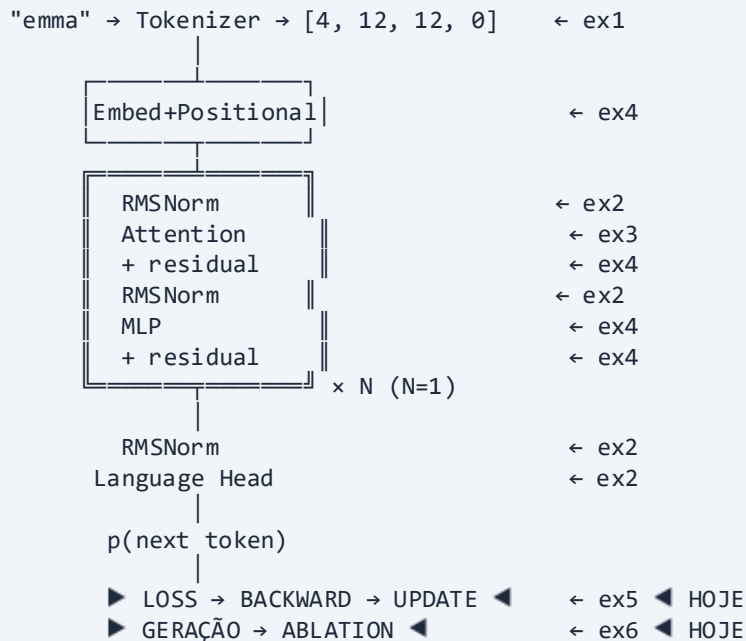
Da Loss à Geração

Semana 3: self-attention, modelo completo (ex3, ex4).

Hoje: ligar a ignição — treinar o modelo e gerar texto.

SGAIM · 2025/2026 · MEIC · ISEL

O mapa do Transformer — onde estamos hoje



HOJE

ex5 — Treino

Loss + SGD + Adam

ex6 — Geração

Temperatura + Ablation

Semanas anteriores

S1 $p(x)$, manifolds

S2 Gradientes + building blocks

S3 Attention + modelo completo

Progresso: ex3 e ex4

Sondagem rápida:

- 1 Quem terminou o ex3 (todos os testes passam)?
- 2 Quem terminou o ex4?
- 3 Quem fez o EXPLORE (remover sqrt(d), remover residual)?

Se ainda estão a acabar o ex3/ex4: **completem esta semana** — são pré-requisito para o ex5.

Hoje: **ex5 (treino) + ex6 (geração + ablation)**. Entrega do MP1: semana 5.

O que optimizamos?

O modelo recebe "emm" e prevê a distribuição sobre o próximo token.

Se o token correcto é "a", queremos que $p(\text{"a"})$ seja alta.

$$L = -\log p(\text{token correcto})$$

Modelo confiante

$$p = 90\% \rightarrow -\log(0.9) = 0.105$$

Loss baixa — bom!

Modelo perdido

$$p = 1\% \rightarrow -\log(0.01) = 4.6$$

Loss alta — mau!

Quanto mais confiante no token certo, menor a loss.

THINK

O modelo acabou de ser inicializado — todos os pesos são aleatórios.

O vocabulário tem 27 tokens (a–z + BOS).

Que loss esperamos no primeiro forward pass?

Pensem 1 minuto.

O ponto de partida: 3.30

3.30

Com pesos aleatórios, o modelo atribui probabilidade uniforme:

$$p = 1/27 \rightarrow L = -\log(1/27) = \log(27) \approx 3.30$$

Checkpoint zero: se a primeira loss for muito diferente de 3.30, algo está errado.

Mesmo raciocínio da aula 01 (MNIST: $-\log(1/10) \approx 2.30$).

Objectivo: fazer a loss descer de 3.30 para valores úteis (~2.0 ou menos).

O loop de treino

```
for epoch in range(n_epochs):  
    for name in dataset:  
        loss = compute_loss(model, name) # forward pass  
        loss.backward() # gradientes  
        optimizer_step(model.parameters()) # actualizar pesos
```

4 passos, repetidos milhares de vezes:

1

Forward

passar tokens pelo modelo →
logits → loss

2

Backward

propagar gradientes (o motor
do ex1)

3

Update

ajustar pesos na direcção que
reduz a loss

4

Repeat

até a loss convergir

No ex5: TODO 1 = forward + loss. TODOs 2–3 = update (SGD, Adam).

SGD: o otimizador mais simples

$$\theta \leftarrow \theta - \eta \cdot \nabla L$$

```
p.data -= learning_rate * p.grad  
p.grad = 0    # reset para a próxima iteração
```

Problemas do SGD:

- ✗ Gradientes ruidosos → oscilações
- ✗ Mesma learning rate para todos os parâmetros
- ✗ Fica preso em vales estreitos

Resumo SGD

- Para cada parâmetro: subtrair $lr \times$ gradiente
- Depois: reset gradiente a zero
- Simples mas limitado
- No ex5 TODO 2: 2 linhas de código

Adam: momentum + learning rates adaptativas

Contador	O que guarda	Efeito
m (momentum)	Média móvel do gradiente	Suaviza direcção — menos oscilação
v (RMS)	Média móvel do gradiente ²	Adapta o passo — grad grandes → passos menores

$$m \leftarrow \beta_1 m + (1 - \beta_1) g$$

$$v \leftarrow \beta_2 v + (1 - \beta_2) g^2$$

$$\theta \leftarrow \theta - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

$\hat{m}$ e $\hat{v}$ são **bias-corrected** (dividir por $1 - \beta^t$) — sem isto, os primeiros passos são enviesados.

No ex5 TODO 3: ~8 linhas. O optimizador mais usado em deep learning.

THINK

Imaginem dois parâmetros:

- Parâmetro A: gradientes oscilam entre $+10$ e -10
- Parâmetro B: gradientes consistentes de $+0.1$

Com SGD (mesma learning rate para ambos), o que acontece?
E com Adam?

Pensem 1 minuto.

Momentum suaviza, RMS adapta

SGD

Parâmetro A

Oscila violentamente ($lr \times \pm 10$)

Parâmetro B

Avança lentamente ($lr \times 0.1$)

Adam

Parâmetro A

Momentum cancela oscilações ($m \approx 0$).
RMS é grande $\rightarrow$ passo pequeno.

Parâmetro B

Momentum mantém direcção ($m \approx 0.1$).
RMS é pequeno $\rightarrow$ passo relativo maior.

Adam adapta automaticamente o tamanho do passo a cada parâmetro. Não é uma learning rate — são milhares de learning rates, uma por parâmetro.

No REFLECT do ex5: vão comparar curvas SGD vs Adam. A diferença é visível.

De logits a texto

Depois de treinar, o modelo produz logits $\rightarrow$ softmax $\rightarrow$ probabilidades.

"emm" $\rightarrow$ modelo $\rightarrow$ $p = [0.01, 0.85, 0.02, \dots, 0.01]$
 a z BOS

O token "a" tem 85% de probabilidade $\rightarrow$ **provável próximo token.**

Processo de geração:

1

Amostrar da distribuição:
escolher token proporcionalmente
à sua probabilidade

2

Token gerado torna-se input:
alimentar de volta ao modelo

3

Repetir até BOS:
BOS = fim da sequência
(fim do nome gerado)

Temperatura: controlar diversidade

$$p_i = \text{softmax}(\text{logits} / T)$$

T	Efeito	Resultado
0.1	Aguça a distribuição	Quase determinístico — sempre o token mais provável
1.0	Sem alteração	Distribuição original do modelo
2.0	Achata a distribuição	Mais aleatório — tokens menos prováveis ganham peso

No ex6 TODO 1: uma linha — dividir cada logit pela temperatura.

THINK

Se $T \rightarrow 0$ (ex: 0.001), o modelo gera sempre o quê?

Se $T \rightarrow \infty$ (ex: 100.0), o modelo gera o quê?

Pensem em termos do que o softmax faz com os logits divididos.

Pensem 1 minuto.

Temperatura na prática

$$T \rightarrow 0$$

$\text{softmax}([\text{logits}/0.001]) \rightarrow$ quase one-hot.
O modelo gera sempre a mesma sequência.
Determinístico, repetitivo.

$$T \rightarrow \infty$$

$\text{softmax}([\text{logits}/100]) \rightarrow$ uniforme.
O modelo escolhe tokens aleatoriamente.
Output = lixo.

O sweet spot: 0.3–1.0, dependendo da tarefa:

Nomes

$$T \approx 0.5$$

coerentes mas variados

Texto criativo

$$T \approx 0.8-1.0$$

mais surpresa

Código

$$T \approx 0.2$$

determinístico, preciso

No ex6 experiment 1: gerar nomes a $T = 0.1, 0.5, 1.0, 2.0$ e comparar.

Ablation: o método científico

Ablation study: remover um componente do modelo, treinar, e medir o impacto.

1 PREDICT

Antes de correr: o que esperam que aconteça? Porquê?

2 RUN

Correr a experiência, registar resultados.

3 EXPLAIN

O resultado confirma ou refuta a previsão? Porquê?

A previsão é tão importante como o resultado. Uma previsão errada bem explicada vale mais que um resultado correcto sem explicação.

As explicações do ex6 valem metade da nota do MP1.

Roteiro hands-on: ex5 e ex6

Ordem: ex5 primeiro (treino), depois ex6 (geração + ablation).

O ex5 treina o modelo que o ex6 vai usar para gerar texto.

ex5 – Treino

3 TODOs:

TODO 1 – Cross-entropy loss

TODO 2 – SGD step

TODO 3 – Adam step

ex6 – Geração + Ablation

3 TODOs – geração:
temperatura, amostragem, EOS

5 experiments – ablation:
PREDICT → RUN → EXPLAIN

Trabalhem em pares. Referência: diagrama do loop de treino (S7).

MP1 entrega: semana 5. Hoje é a última aula antes da entrega.

Ex5 — O que implementar

TODO	O quê	Dica
1	Cross-entropy loss	Para cada posição: feed $\rightarrow$ logits $\rightarrow$ softmax $\rightarrow -\log(\text{prob}[\text{target}])$. Média.
2	SGD step	$p.\text{data} -= lr \times p.\text{grad}$; $p.\text{grad} = 0$ — 2 linhas
3	Adam step	m , v , bias correction, update — ~8 linhas (seguir fórmulas do S9)

Verificação:

- ✓ Loss inicial ≈ 3.30
- ✓ Loss desce durante o treino
- ✓ Loss final < 2.5

Se a loss inicial for muito diferente de 3.30, o TODO 1 tem um bug.

Ex6 Parte A – Geração

TODO	O quê	Dica
1	Temperatura	Dividir cada logit por T antes do softmax – 1 linha
2	Amostragem	<code>random.choices(range(vocab_size), weights=probs)</code>
3	Deteção de fim	Se <code>token == BOS</code> → <code>break</code> . Senão, <code>append</code> ao nome.

Depois dos TODOs: gerar 10 nomes a $T=0.5$ e verificar que são strings válidas.

Ex6 Parte B – Ablation experiments

5 experiências, cada uma com PREDICT → RUN → EXPLAIN:

#	Experiência	O que remover/mudar
1	Temperatura	Gerar a T = 0.1, 0.5, 1.0, 2.0
2	Sem residual	Remover residual connections → treinar → loss?
3	Sem RMSNorm	Remover normalização → treinar → estável?
4	1 head vs 4 heads	n_head=1 (dim 16) vs n_head=4 (dim 4)
5	Contexto curto	block_size=4 vs block_size=16

Deliverable: para cada experiment, 2–3 frases com previsão + observação + explicação. **Código que não sabem explicar → nota baixa.**

Como os LLMs reais aprendem

Acabaram de treinar o vosso modelo em 32k caracteres de nomes.

O GPT-4 treina em triliões de tokens. Mas o processo tem a **mesma base** — e mais duas fases:

Fase 1 Pre-training

Estudar manuais
(ler tudo, prever a próxima palavra)

← *vocês fizeram isto (ex5)*

Fase 2 SFT

Resolver exercícios resolvidos
(Q&A com respostas humanas)

Fase 3 RLHF

Ter apenas a resposta certa,
sem resolução (preferências)

Analogia de Karpathy (2023)

O vosso microGPT só faz a Fase 1. Mas é a base de tudo o resto.

Fase 1: Pre-training

$$L = -(1/N) \sum \log p(t_i | t_1, \dots, t_{i-1})$$

É exactamente o que implementaram no ex5 TODO 1 — mas **à escala**:

	microGPT	GPT-4 (estimado)
Dados	32k chars (nomes)	~13 triliões de tokens
Parâmetros	4,192	~1.8 triliões
Compute	1 CPU, segundos	milhares de GPUs, meses
Loss function	✓ mesma	✓ mesma
Arquitectura	✓ mesma	✓ mesma

O modelo aprende gramática, factos, raciocínio — tudo como efeito secundário de prever a próxima palavra.

Fases 2 e 3: SFT + RLHF

SFT (Supervised Fine-Tuning)

O modelo pré-treinado "fala" mas não "conversa". SFT treina-o em pares pergunta→resposta curados por humanos. Aprende o formato de assistente.

RLHF (Reinforcement Learning from Human Feedback)

Humanos comparam duas respostas e dizem qual preferem. O modelo aprende a gerar respostas preferidas — sem ver a resposta "correcta".

Pre-training: "O gato sentou-se no ____" → prever "tapete"
SFT: "O que é um gato?" → "Um gato é um mamífero..."
RLHF: Resposta A vs B → humanos preferem B → reforçar B

O vosso microGPT só faz a Fase 1. Mas é a base de tudo o resto.

Scaling laws + a ponte

Scaling laws (Kaplan et al., 2020): a loss desce de forma previsível com mais dados, mais parâmetros, mais compute. A relação é uma **power law**.

O que é igual

✓ Arquitetura (attention, residual, MLP, norm)

✓ Loss function (cross-entropy)

✓ Treino: next-token prediction

✓ Adam como otimizador

O que muda

→ Escala: dims, layers, vocab

→ Dados: 32k chars → triliões de tokens

→ Compute: 1 CPU → milhares de GPUs

→ Post-training: SFT + RLHF

**O vosso modelo de 4,192 parâmetros é algoritmicamente idêntico ao GPT-4.
A diferença é escala e engenharia.**

Setup Ollama (demo, se houver tempo)

O mesmo motor que construíram — à escala real.

```
# Instalar Ollama (ollama.com)
ollama run llama3.2:1b # modelo de 1B parâmetros
```

microGPT

4,192

parâmetros, CPU, segundos



Llama 3.2

1B

parâmetros, GPU/CPU, tempo real

Attention, residual, MLP, softmax, temperatura — tudo o mesmo. Só a escala muda.

TPC: instalar Ollama no portátil e correr um modelo. Semana 5 vai usar Ollama extensivamente.

O que construíram em 4 semanas

Semana	O quê	Exercícios
1	$p(x)$, manifolds, distribuições	—
2	Motor de gradientes + building blocks	ex1, ex2
3	Attention + modelo completo	ex3, ex4
4	Treino + geração + ablation	ex5, ex6

De zero a um GPT completo — construído por vocês, peça a peça.

MP1: o que entregar (semana 5)

6 exercícios com todos os testes a passar.

Mas o código não é o deliverable principal.

O deliverable são as explicações do ex6: para cada ablation experiment, previsão + resultado + explicação. 2–3 frases por experiment — claras, concisas, fundamentadas.

Código que não sabem explicar → nota baixa.

Explicação profunda com código parcial → nota mais alta.

Data de entrega: início da semana 5.

Semana 5 + TPC

TPC

- Completar ex5 e ex6 (todos os testes, todas as explicações)
- Instalar Ollama (ollama.com) e correr: `ollama run llama3.2:1b`

Na próxima semana

- **Entrega do MP1**
- Tema: O LLM é só uma peça — stateless, sem ferramentas, sem memória
- Comparação de modelos locais com Ollama (1B, 3B, 7B)
- Prompt engineering: zero-shot, few-shot, chain-of-thought

De construtores do motor a utilizadores do veículo.